CAPSTONE PROJECT BRIEFING

Cloud-Native Application Deployment, CI/CD Automation & Observability

1. Project Objective

The objective of this Capstone Project is to design, implement, deploy, and demonstrate a complete cloud-native application infrastructure using the technologies and practices covered during the course.

Each student must demonstrate practical knowledge of application deployment, containerization, CI/CD automation, Kubernetes or serverless architecture, infrastructure monitoring, and automated scaling.

This is an individual, hands-on implementation project, not just a documentation or presentation assignment. Every student must build and demonstrate their own solution. All major components must be deployed, functional, and ready for a live demonstration.

2. Project Requirements

A. Project Documentation and Architecture

Each student must prepare technical documentation that includes:

- **Application idea:** Explain the problem being solved, the application's purpose, its key features, and its intended users.
- **Architecture diagram:** Provide a clear architecture drawing showing the application's components, deployment architecture, and flow of traffic and data.
- **Deployment architecture:** Use either a Kubernetes cluster for containerized application deployment or a serverless architecture, where appropriate.
- **CI/CD architecture:** Illustrate how a GitHub push trigger initiates the pipeline, builds a new container image, and pushes it to Docker Hub.
- **Notification system:** Show how a successful pipeline execution sends a notification to Slack, Microsoft Teams, or WhatsApp.
- **Monitoring architecture:** Include Prometheus and Grafana and explain how metrics are collected, stored, and visualized.

The documentation must explain the purpose of each component and how the components interact with one another.

B. Actual Implementation and Working Infrastructure

Each student must implement and demonstrate the following working components:

- A GitHub repository containing the application source code and required configuration files.
- A working CI/CD pipeline configured with a push trigger.
- Automated application build and container image creation.
- A Docker Hub repository containing the built container images.
- A working deployment environment using Kubernetes or the approved serverless architecture.
- A notification integration using Slack, Microsoft Teams, or WhatsApp.
- A working Prometheus and Grafana monitoring setup.

Students must be able to explain their configuration, demonstrate the workflow, and troubleshoot basic issues.

C. Monitoring and Observability

The project must include a functional monitoring and observability setup using **Prometheus and Grafana**. The monitoring solution must cover the infrastructure and application deployment environment, including:

- CPU and memory utilization.
- Kubernetes node and cluster resource utilization, where applicable.
- Pod status, pod resource consumption, and restart information.
- Application or workload performance metrics, where available.
- Horizontal Pod Autoscaler (HPA) metrics and replica counts.
- A Grafana dashboard displaying the relevant metrics.

Prometheus must collect the required metrics, and Grafana must display them in a usable dashboard. Students must explain what the metrics mean and how the dashboard helps identify resource pressure, workload changes, and scaling activity.

D. Live Demonstration: Horizontal Pod Autoscaling

Each student must conduct a live demonstration of their working application and demonstrate automatic scaling under load.

- Access the deployed application and show that it works.
- Explain the Horizontal Pod Autoscaler (HPA) configuration, including the target metric and minimum and maximum replica counts.
- Run a stress test against the application to generate workload.
- Show the HPA detecting increased resource utilization.
- Demonstrate the automatic scaling out of application pods.
- Show the increased pod count and relevant resource metrics on the Grafana dashboard.
- Stop the stress test and observe the scaling behaviour as the workload decreases, where applicable.

Important: Scaling must happen automatically in response to the workload. Manually increasing the replica count does not satisfy the HPA demonstration requirement.

The application, stress test, Kubernetes metrics, HPA, and Grafana dashboard must work together during the demonstration.

3. Mandatory Deliverables

- Project documentation.
- Application architecture diagram.
- CI/CD pipeline architecture diagram.
- GitHub repository.
- Docker Hub repository.
- Working CI/CD pipeline with push trigger.
- Successful pipeline notification integration.
- Deployed application and working infrastructure.
- Prometheus and Grafana monitoring setup.
- Grafana dashboard showing relevant infrastructure and scaling metrics.
- Working HPA configuration and stress-test demonstration.

- Final presentation with a live demonstration.

4. Final Presentation and Evaluation

During the final presentation, each student must explain their application, walk through the architecture, demonstrate the CI/CD pipeline, and show the deployed application and monitoring dashboards.

The live demonstration must include a fresh code push, automated container image build and push, pipeline notification, and HPA-based scaling under load.

Students should be prepared to explain their design decisions, demonstrate that the components work together, and answer technical questions.

5. Evaluation Criteria

- **Documentation and architecture:** Clarity, completeness, and sound design decisions.
- **CI/CD automation:** Correct push-triggered execution, image build, Docker Hub push, and notifications.
- **Deployment and functionality:** Successful deployment and operation of the application.
- **Monitoring and observability:** Correct Prometheus metric collection and useful Grafana dashboards.
- **Autoscaling demonstration:** Successful load generation, automatic pod scaling, and visualization of scaling activity.
- **Technical understanding:** Ability to explain the architecture, configurations, workflow, and troubleshooting approach.
- **Presentation and live demo:** Clear communication and successful demonstration of the complete workflow.

6. Final Expectations

The Capstone Project should represent a complete, working cloud-native delivery workflow, from source code to deployment, monitoring, and scaling.

Each student is expected to build and demonstrate their own solution. A diagram, screenshots, recorded video, or theoretical explanation alone will not substitute for the required working implementation and live demonstration.

The ultimate goal is to demonstrate that you can build, deploy, monitor, and scale a real application using modern DevOps practices.

Suggested End-to-End Demonstration Flow

1. Push new code to GitHub

Trigger the automated workflow.

2. CI/CD pipeline executes

Build the application, create a container image, and push it to Docker Hub.

3. Send a success notification

Notify the team through Slack, Teams, or WhatsApp.

4. Run the deployed application

Verify the updated application is working in the target environment.

5. Generate load and observe scaling

Run a stress test, watch HPA scale out the pods, and verify the changes on Grafana.

Technical note: For Kubernetes projects, configure a metrics source for HPA, such as Metrics Server for CPU-based scaling, alongside Prometheus for monitoring and Grafana for visualization. Prometheus and Grafana alone do not automatically enable CPU-based HPA scaling.